POLITECNICO DI TORINO

Robotics Project – 2025/2026

Robotics Project

LLM-Guided Pick-and-Place Manipulation with YOLO Perception, Waypoint Optimisation and Adaptive Gripper Control

Group number:
42

Students:
Sergio Andres Gutierrez de Piñeres Ramirez
David Santiago Alfonso Montanez
Daniel Felipe Cadavid Osorio
Jossua Esteban Ruiz Pajaro
Yanjun Chen

A.Y. 2025/26

1 Understanding of the Reference Paper

The reference paper *Language Models as Zero-Shot Trajectory Generators* studies whether a Large Language Model (LLM) can generate robot end-effector trajectories from natural-language instructions and visual object information.The problem addressed by the paper is that most previous LLM-based robotic systems use the LLM mainly as a high-level planner, while the actual motion is produced by predefined skills, motion primitives or trajectory optimisers. [1]

The paper challenges this assumption by asking whether the LLM can generate the robot motion itself. The LLM does not directly control the robot motors. Instead, the system provides perception and robot-execution APIs. The vision system detects the objects and provides their spatial information, the LLM generates executable motion code and trajectory points, and the robot controller executes the generated end-effector poses. The original work uses a segmentation-based vision pipeline and evaluates the method on different manipulation tasks, including pick-and-place, wiping, shaking and drawing. Our project follows the same perception-to-motion idea, but adapts it to our ROS1/Gazebo Sawyer platform and focuses on pick-and-place manipulation.

2 Solution Proposal and Implementation Methodology

Our proposed solution is an LLM-guided pick-and-place pipeline that connects natural-language commands, visual perception and robot execution. The complete workflow is designed to show the working logic of the system rather than the source code.

At the beginning of the program, the Sawyer robot starts from a predefined safe parking pose. Before executing any user task, the robot first moves to a top observation pose. This step is important because the wrist-mounted camera needs a clear view of the workspace before the target object positions can be estimated.

For perception, we use YOLO instead of the vision method used in the reference paper. The original paper relies on a more complex segmentation-based pipeline, which was difficult to reproduce within the limited project time and with our available setup. Therefore, we chose YOLO because it is easier to train or configure for a small set of predefined objects, faster to integrate with ROS1, and sufficient for the pick-and-place tasks considered in our project. This choice reduced the implementation time while still allowing the system to obtain object positions from camera observations.

After the robot reaches the observation pose, the wrist-mounted camera captures RGB and depth data. YOLO detects the predefined objects in the image. Then, using depth information, camera intrinsics and the camera pose, the detected object positions are transformed from the camera frame to the Gazebo world frame. The result is stored as a visual scene dictionary. This scene contains the current end-effector pose, detected object names, object centres, base positions, approximate half-extents, confidence values, table height and workspace limits.

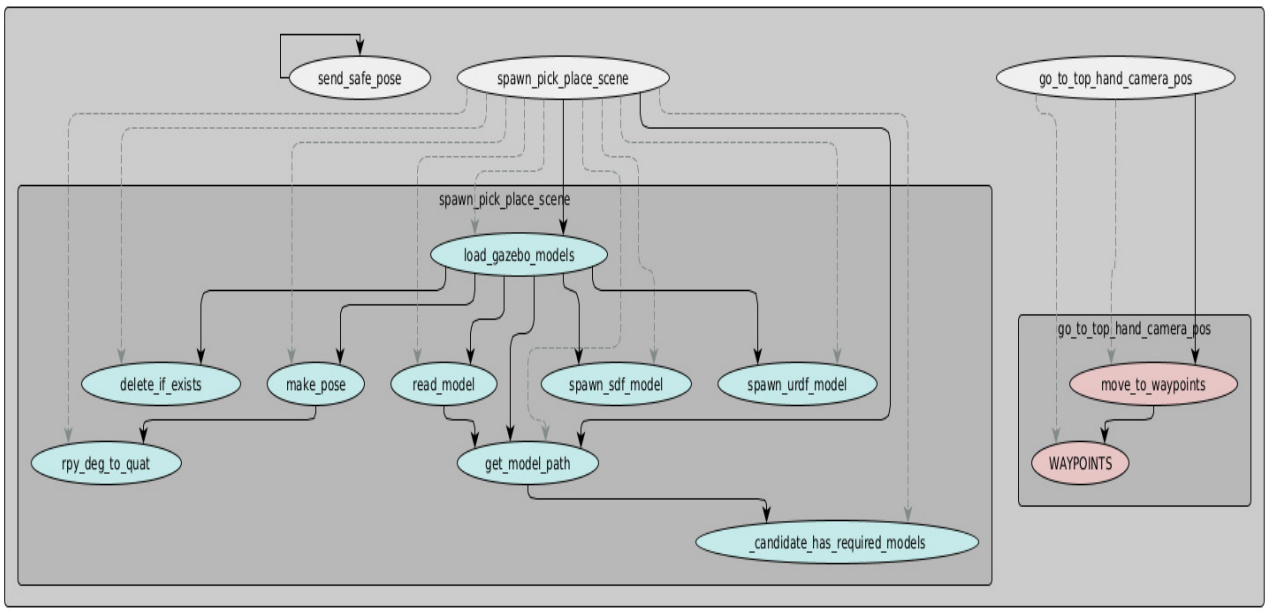

Figure 1: Software architecture and function dependency graph. Solid arrows represent function calls, while dashed arrows represent module membership or definitions.

The LLM is called in `run_task.py`. The system prompt defines the available robot APIs and the rules for generating safe manipulation motions. Given a natural-language command, the LLM generates an executable Python function called `execute_task()`. The generated function first calls `make_scene()` to obtain the visual scene. Then it uses the object positions in the scene to generate a sequence of manipulation waypoints. Each waypoint contains a target position, orientation and gripper command:

$$\{\texttt{pos} : [x, y, z], \quad \texttt{ori} : [roll, pitch, yaw], \quad \texttt{gripper} : \texttt{open/close}\}.$$

A typical waypoint sequence includes moving above the object, descending to the grasping position, closing the gripper, lifting the object, moving above the target plate, descending to the placement pose, opening the gripper and returning to the safe pose. Before execution, the generated waypoints are passed to `adjust_grasp_waypoints(scene, waypoints)`. This function refines the LLM-generated points using the detected scene information. Finally, `execute_waypoints(waypoints)` sends the corrected motion to the robot. The execution layer performs inverse kinematics, separates horizontal and vertical motion when necessary, controls the gripper and returns the robot to the safe pose after completion or failure.

3 Implementation Choices and Innovation

Our implementation was completed entirely in simulation. Instead of reproducing the real-world robotic setup used in the reference paper, we built our own ROS1/Gazebo environment with a Sawyer robot and a set of predefined objects. This choice made the project more feasible within the available time, while still allowing us to validate the complete perception-to-motion pipeline.

For the manipulation scene, we created ten custom objects and wrote their corresponding SDF files. These models were loaded into Gazebo and used as the objects detected by the perception system and manipulated by the robot. By defining our own objects, we could control their size, shape and position, which made it easier to test the grasping and placing behaviours in a repeatable simulation environment.

Compared with the full range of tasks studied in the reference paper, our implementation focuses only on the grasping and placing part. The original paper includes more complex actions such as wiping, shaking, drawing and opening objects. In this project, we selected pick-and-place manipulation as the target task because it is the most fundamental operation for validating the integration of vision, LLM-generated motion and robot execution.

The first implementation choice is the use of YOLO for perception. The reference paper uses a more complex segmentation-based visual pipeline, which was difficult to reproduce with our available setup and project time. YOLO provided a practical alternative because it can be trained or configured for our custom objects, integrates well with ROS1, and is sufficient for estimating object centres in pick-and-place tasks.

The innovation is the gripper feedback and adaptive closing strategy. Instead of only sending a close command and assuming that the grasp succeeds, the system monitors the gripper opening distance during closure. When the gripper is far from the expected object width, it can close faster. When it approaches the object, the closing speed is reduced. A successful grasp is detected when the opening becomes stable while remaining above the empty-gripper closing limit.

4 Results and Discussion

The system was tested in the ROS1/Gazebo environment with several predefined objects. The robot was able to move to the observation pose, detect objects using YOLO, build the visual scene, let the LLM generate task code, optimise the generated waypoints and execute pick-and-place motions.

The results show that the complete pipeline can connect natural-language commands with visual perception and robot execution. The LLM provides the main waypoint sequence, while the optimisation layer improves grasping and placing accuracy. The gripper feedback mechanism increases reliability by checking whether the object is actually held after closing.

In our experiments, **10** trials were tested. YOLO correctly detected the target object in **10** cases. The robot successfully grasped the object in **7** cases and successfully placed it on the target plate in **6** cases. Failed grasps were correctly detected in **8** cases.

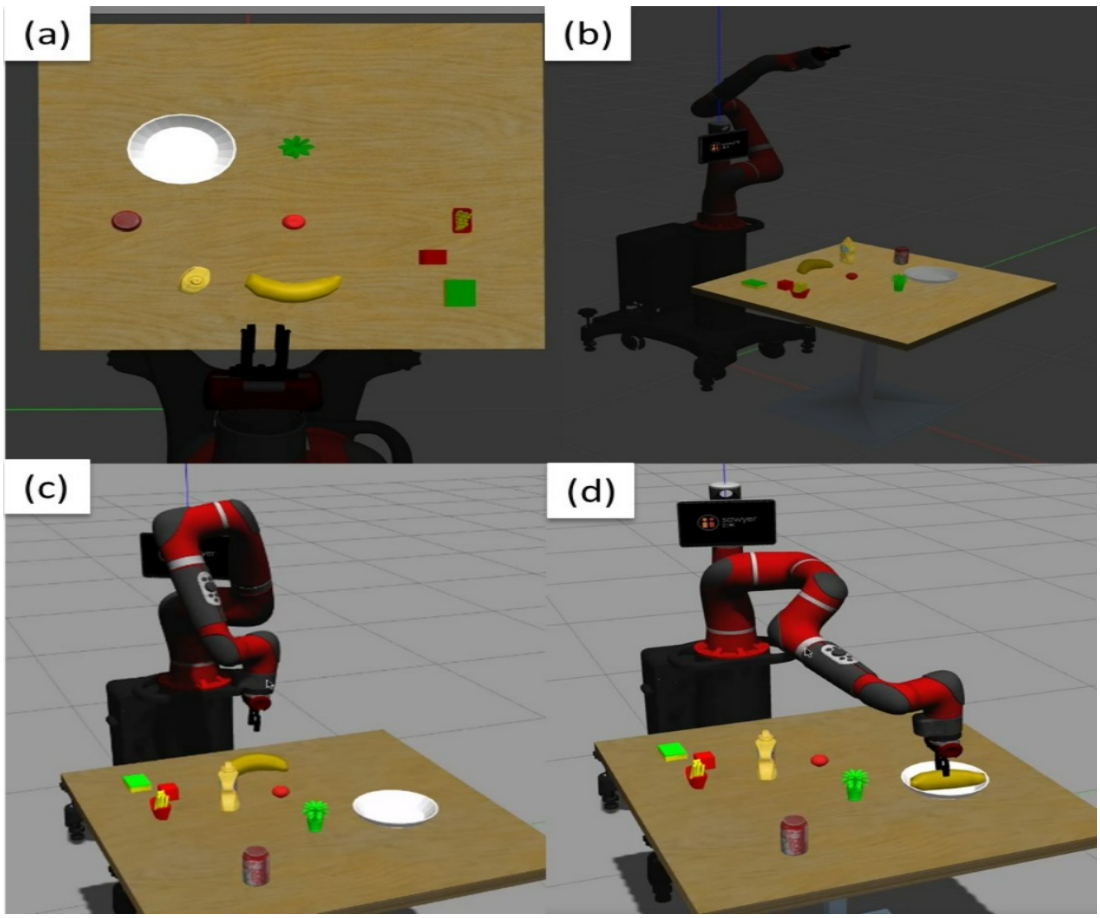

Figure 2: Vision-Based Object Detection and Pick-and-Place Execution in the Sawyer Robot Simulation.(a) Object detection from the hand camera. (b) The Sawyer robot moves to the top observation pose. (c) The robot moves to a safe position before task execution. (d) The robot places the banana onto the target plate.

5 Limitations and Future Work

Continuous visual feedback. The current system mainly localises the objects once, before execution starts. If the target object moves after the initial observation, the stored coordinates may become outdated. A future improvement is to repeat object detection before grasping or to add continuous visual feedback during execution.

Tactile sensing on the gripper. At present, grasp feedback is mainly based on the gripper opening distance. Adding tactile or force sensors on the gripper fingers would provide clearer information about contact, grasp stability and excessive force. This would make the adaptive gripper-control strategy more reliable.

Extension to more complex tasks. Our implementation focuses on pick-and-place manipulation. Future work could extend the system to tasks closer to the reference paper, such as wiping, shaking, drawing, rotating objects or opening caps.

6 Conclusion

This project implemented an LLM-guided robotic manipulation system using ROS1, YOLO perception, a wrist-mounted camera and a Sawyer robot. The system follows the key idea of the reference paper: visual information provides object positions, and the LLM generates executable robot-motion code. Our main contributions are the optimisation of LLM-generated waypoints and the gripper-distance-based adaptive control strategy. These additions improve motion reliability, grasp checking and safe execution in a simulation-based pick-and-place pipeline.

References

[1] T. Kwon, N. Di Palo, and E. Johns, “Language Models as Zero-Shot Trajectory Generators,” *IEEE Robotics and Automation Letters*, 2024.